2. Entregable 02 · Mapa de dominio (DDD)

2.1 Lenguaje ubicuo (glosario → código)

Término del glosario	Nombre en el modelo	Tipo DDD
Venta	`Venta`	Aggregate Root
(renglón de la venta)	`LineaVenta` (propuesto)	Entity
Comprobante / Folio	`Comprobante`, `Folio`	Aggregate Root / Value Object
Apartado, Anticipo, Vencimiento de Apartado	`Apartado`, `Anticipo`, `FechaVencimiento`	Aggregate Root / VO / VO
Devolución, Reembolso, Plazo de Devolución	`Devolucion`, `Reembolso`, `PlazoDevolucion`	Aggregate Root / VO / VO
Inventario, Disponible, Reservado	`StockSucursal` (`disponible()`, `reservado`)	Aggregate Root
Número de Serie	`NumeroDeSerie`	Aggregate Root (unicidad global)
Diferencia/Faltante	`ReporteFaltante`	Aggregate Root
(movimiento entre sucursales, US16)	`TransferenciaInventario` (propuesto)	Aggregate Root
Producto	`Producto` (con `PoliticaDevolucion`)	Aggregate Root (catálogo)
Cliente	`Cliente`	Aggregate Root
Vendedor, Cajero, Supervisor, Almacén	`Empleado` con `Rol`	Aggregate Root / VO
Sucursal	`Sucursal`	Aggregate Root
Método de Pago	`MetodoPago` (Efectivo, Tarjeta, Transferencia) y `Pago`	Value Objects
Auditoría	`RegistroAuditoria`	Aggregate (append-only)
Garantía	Fuera del dominio: solo se registra el serial (glosario #24)	—

2.2 Bounded Contexts

Contexto	Responsabilidad	Aggregates	Tipo
Ventas y Caja	Venta, pagos, comprobantes	Venta, Comprobante	Core
Apartados	Reserva con anticipo y vencimiento	Apartado	Core
Devoluciones	Políticas de devolución, reembolso/cambio	Devolucion	Core
Inventario y Seriales	Existencias por sucursal, seriales, transferencias, faltantes	StockSucursal, NumeroDeSerie, TransferenciaInventario, ReporteFaltante	Core
Catálogo y Maestros	Productos, clientes, empleados, sucursales	Producto, Cliente, Empleado, Sucursal	Supporting
Auditoría	Bitácora inmutable de operaciones	RegistroAuditoria	Generic

Los contextos se comunican por **identificadores** (SKU, Folio, ClienteId) y por **Domain Events**, nunca por referencias directas entre Aggregates.

2.3 Aggregates, Entidades, Value Objects

Aggregate Root	Entidades internas	Value Objects	Invariantes que protege
`Venta`	`LineaVenta`	`Folio`, `Dinero`, `Pago`, `MetodoPago`, `DatosPagoElectronico` (últimos 4 dígitos, banco, referencia)	Todo producto con `RequiereSerial` lleva serial en su línea (100% de seriales registrados); monto pagado = total; vencimiento de devolución = fecha de venta + 30 días
`Apartado`	—	`Anticipo`, `FechaVencimiento`, `Dinero`	Anticipo ≥ 25% del precio total; vencimiento = creación + 14 días (no configurable); estados Activo → Completado / Cancelado / Vencido

Aggregate Root	Entidades internas	Value Objects	Invariantes que protege
`Devolucion`	—	`PlazoDevolucion`, `Reembolso`, `Cambio`, `AprobacionSupervisor`	Requiere comprobante O aprobación de Supervisor; dentro de 30 días y sin daño; reembolso por el método de pago original
`StockSucursal`	—	`Cantidad`	disponible = total − reservado − dañado − en tránsito; nunca negativo; un producto RESERVADO no se vende
`NumeroDeSerie`	`EventoSerial` (historial)	`ValorSerie`	Valor único en todo el sistema; un serial no puede estar en dos ventas a la vez; estados Disponible/Vendido/Devuelto/Dañado
`Comprobante`	—	`Folio`, `TipoComprobante`	Folio único y secuencial; tipo Venta/Apartado/Devolución
`TransferenciaInventario` (propuesto)	—	`Cantidad`	Cantidad bloqueada "en tránsito" en origen; la autoriza un Supervisor
`ReporteFaltante`	—	`Cantidad`	Diferencia = cantidad_sistema − cantidad_física; el ajuste requiere aprobación de Supervisor (faltantes > $1000)

2.4 Domain Services y Domain Events

Domain Services (lógica que no pertenece a un solo Aggregate): `PoliticaDevolucionService` (plazo, comprobante, daño, umbral $500), `CalculadoraAnticipo` (25%), `GeneradorFolio` (secuencial único), `ConciliadorInventario` (faltantes, umbral $1000).

Domain Event	Lo emite	Reacciona	Origen
`VentaRegistrada`	Venta	Stock descuenta; Comprobante se emite; Auditoría registra	US02
`SerialAsignado`	NumeroDeSerie	Auditoría; historial del serial	US03, US11
`ApartadoCreado`	Apartado	Stock reserva unidades; Comprobante tipo Apartado	US04
`ProductoReservado` / `ProductoLiberado`	StockSucursal	Bloquea o libera venta	US05
`ApartadoCompletado`	Apartado	Se crea Venta formal con nuevo folio; se libera RESERVADO	US06
`ApartadoVencido` / `ApartadoCancelado`	Apartado (batch fin de día)	Libera stock; notifica al cliente; Auditoría	US07
`DevolucionSolicitada` / `DevolucionRechazada` / `DevolucionAprobada`	Devolucion	Registro de motivo; Auditoría	US08, US09
`DevolucionProcesada`	Devolucion	Stock suma unidad; serial pasa a Devuelto; Comprobante tipo Devolución	US08
`DiferenciaDetectada`	ReporteFaltante	Tarea de investigación; aprobación de Supervisor	US10
`TransferenciaIniciada` / `TransferenciaRecibida`	TransferenciaInventario	Stock origen/destino	US16

Domain Event	Lo emite	Reacciona	Origen
`ComprobanteEmitido`	Comprobante	Impresión	US12

2.5 Diagrama DDD (código PlantUML)

```
@startuml DDD_TecnoStock
skinparam packageStyle rectangle
skinparam classAttributeIconSize 0
hide empty members

package "Ventas y Caja (Core)" #E8F0FE {
  class Venta <<Aggregate Root>>
  class LineaVenta <<Entity>>
  class Pago <<Value Object>>
  class Comprobante <<Aggregate Root>>
  Venta *-- "1..*" LineaVenta
  Venta *-- "1..*" Pago
}

package "Apartados (Core)" #EAF7EA {
  class Apartado <<Aggregate Root>>
  class Anticipo <<Value Object>>
  class FechaVencimiento <<Value Object>>
  Apartado *-- Anticipo
  Apartado *-- FechaVencimiento
}

package "Devoluciones (Core)" #FDECEA {
  class Devolucion <<Aggregate Root>>
  class AprobacionSupervisor <<Value Object>>
  class Reembolso <<Value Object>>
  class Cambio <<Value Object>>
  Devolucion *-- AprobacionSupervisor
  Devolucion o-- Reembolso
  Devolucion o-- Cambio
}

package "Inventario y Seriales (Core)" #FFF4E0 {
  class StockSucursal <<Aggregate Root>>
  class NumeroDeSerie <<Aggregate Root>>
  class EventoSerial <<Entity>>
  class TransferenciaInventario <<Aggregate Root>>
  class ReporteFaltante <<Aggregate Root>>
  NumeroDeSerie *-- "0..*" EventoSerial
}

package "Catalogo y Maestros (Supporting)" #F3EAFB {
  class Producto <<Aggregate Root>>
  class Cliente <<Aggregate Root>>
  class Empleado <<Aggregate Root>>
  class Sucursal <<Aggregate Root>>
}
```

```
package "Auditoria (Generic)" #EEEEEE {
  class RegistroAuditoria <<Aggregate Root>>
}

Venta ..> StockSucursal : descuenta (VentaRegistrada)
Venta ..> NumeroDeSerie : asigna serial
Apartado ..> StockSucursal : reserva / libera
Apartado ..> Venta : completa (ApartadoCompletado)
Devolucion ..> Venta : referencia por Folio
Devolucion ..> StockSucursal : suma unidad devuelta
Devolucion ..> NumeroDeSerie : estado Devuelto
Venta ..> Comprobante : ComprobanteEmitido
Devolucion ..> Comprobante
Apartado ..> Comprobante
Venta ..> Producto : por SKU
Venta ..> Cliente : por Documento
Venta ..> Empleado : vendedor / cajero
StockSucursal ..> Sucursal
TransferenciaInventario ..> StockSucursal : en transito
ReporteFaltante ..> StockSucursal : ajuste
Venta ..> RegistroAuditoria : eventos
Apartado ..> RegistroAuditoria : eventos
Devolucion ..> RegistroAuditoria : eventos
NumeroDeSerie ..> RegistroAuditoria : eventos
@enduml
```

